Resilient Raspberry Pi OS – der „unkaputtbare“ Raspi



Tux-Tage ~~Live~~ Lite 2026

Stefan Baur, X2Go Lead Evangelist

Vorstellung Stefan Baur







magazin für computer technik

heise online

Make:

Problem: Zu viel Inhalt, zu wenig Zeit

- Mein Projekt „Resilient Raspberry Pi“ hat mittlerweile so viele Branches, zu denen es etwas zu erzählen gibt, dass mir die Vortragszeit nicht für alles reicht → Folien als Nachschlagewerk (alle im git-Repo, Ordner presentation_slides)
- Wir werden uns daher heute nur die Branches/Themen
  - main
  - example
  - x2goserver-kdrive-lxde
  - x2gominiclient
  - minimal

  und

  anschauen – letzteren in Verbindung mit Ansible.

Ausblick: Was gibt's/gab's wo?

- Bereits stattgefundene Vorträge zu diesem Thema:
  - TÜBIX (in Präsenz) – Erste Fassung des Vortrags, grober Überblick
    - Keine Aufzeichnung, aber Slides im Git-Repo
  - Kielux (remote) – Updates, single- und multiport-printserver, Debugging via SPI-Display
    → alle heutigen Folien, die schon beim Kielux gezeigt wurden, sind so markiert: K
    - Aufzeichnung auf YouTube: https://youtu.be/9eaidscfcEI
    - Slides im Git-Repo
  - LinuxDay.AT Dornbirn (in Präsenz) – Themen:
    wifi-to-upstream-lan-accesspoint, USB-Ethernet-Debugging
    - Aufzeichnung hoffentlich demnächst auf
      https://www.linuxday.at/archiv-geschichte
    - Slides im Git-Repo
- Noch keine Zusage: Chemnitzer Linuxtage, 28.03.2027 – 29.03.2027 (in Präsenz) – entweder Docker-Mailcow, Docker-paperless-ngx, Docker-Euro-Office, oder LAN-to-WiFi-Upstream-Bridge, NAS-Proxy, vorschaltpi-v2

Kielux 2026

LinuxDay.AT 2026

Ausschließlich im Kielux-Vortrag

Im Kielux-Vortrag gibt es außerdem noch ein paar Folien, die heute aus Zeitgründen fehlen, und die ihr euch anschauen solltet, bevor ihr ein System nach meinem Konzept aufsetzt. Das sind folgende Folien:

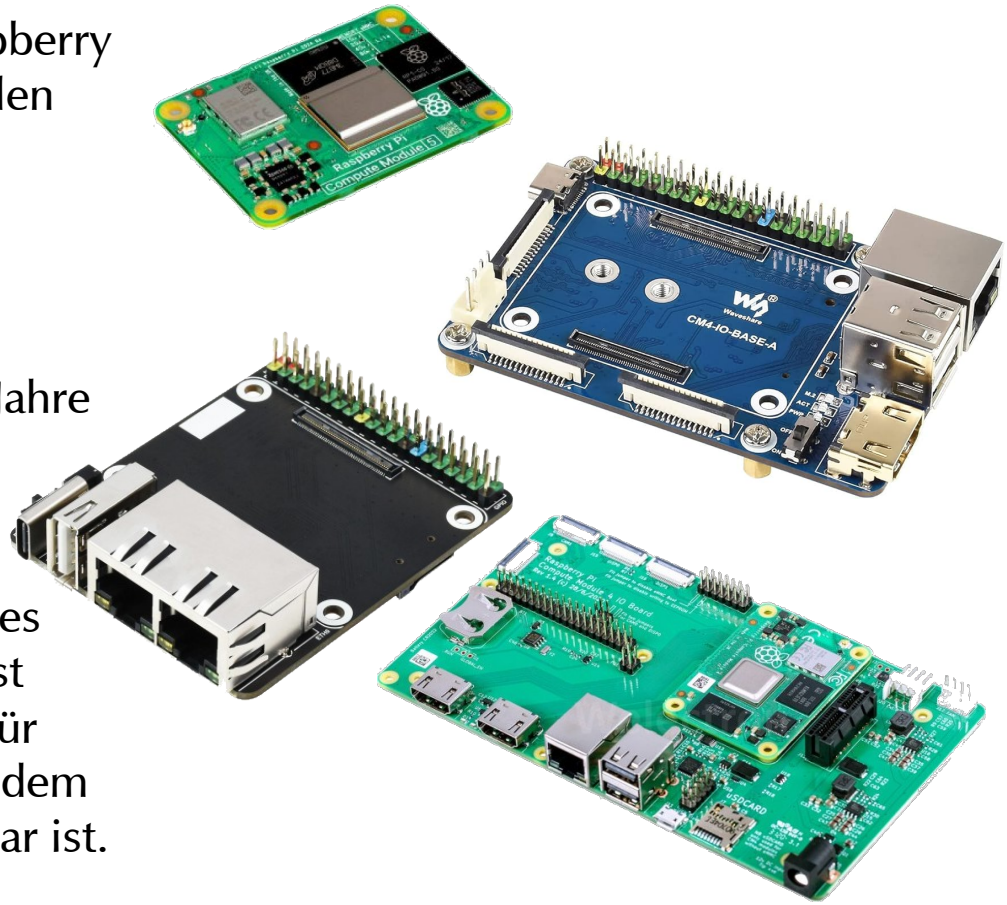
- *Wichtige Hinweise* – für Compute Modules, USB-Sticks, Ansible
- *base_install.conf 1/4* bis *base_install.conf 4/4* – hier wird die Basiskonfiguration ausführlicher besprochen
- *Produktivbetrieb von x2goserver* – erklärt, welche Dinge, ihr beachten solltet, wenn ihr aus dem x2goserver-Branch, der ja nur zur Demo gedacht ist, ein Produktivsystem machen wollt
- *Wie umschalten/updaten?* – erklärt, mit welchen vorgefertigten und im Repo/im installierten System vorhandenen Skripten/Befehlen ihr zwischen den Umgebungen umschalten könnt, am Beispiel von ENV2 nach ENV3
- *Von ENV3 zurück zu ENV2* – erklärt anhand des x2goserver-Branches, wie man aus dem ENV3 dann wieder zurück ins ENV2 kommt, und dabei seine Daten mit umzieht
- *Experten-Skripte* – erklärt, mit welchen vorgefertigten und im Repo/im installierten System vorhandenen Skripten ihr das Overlay-Dateisystem aus-, sowie teilweise oder komplett wieder einschaltet, wenn das z.B. für eine manuelle Installation im aktuell gebooteten System notwendig werden sollte.

„Volksmeinung“ über Raspis

- Nettes Spielzeug, aber nicht für professionellen Einsatz geeignet.
- Zu frickelig/zu instabil:
  - Bei auch nur halbwegs anspruchsvoller Nutzung gehen die microSD-Karten ständig kaputt, außerdem sind Zugriffe träge.
  - Software hängt sich gefühlt „spontan“ auf, man muss ans Gerät und es resetten.
- Entsprechend unwirtschaftlich hoher Wartungsaufwand!

Wie Rio Reiser singt: „Alles Lüge!“

- Zum einen gibt es spezielle Hardware der Raspberry Pi Foundation, die explizit für den professionellen Einsatz gedacht ist – die Compute Modules:
  - Vorteile:
    - auf Dauereinsatz ausgelegt
    - Ersatzteil-Verfügbarkeit auf mindestens 10 Jahre ab Markteinführung garantiert
  - Nachteil:
    - Compute Modules brauchen ein sogenanntes Base-Board – das muss man entweder selbst entwickeln oder hoffen, dass es schon ein für den geplanten Einsatzzweck passendes auf dem Markt gibt, und das auch lang genug lieferbar ist.



(Neudeutsch auch: „Fake News!“)

- Zum anderen sind tatsächlich auch normale Raspberry Pis heute in diversen kommerziellen Produkten zu finden.
- Erkennbar an der charakteristischen Anordnung der Ports, selbst wenn manche Hersteller es nicht offiziell zugeben.
- Mit der Erfahrung aus dem Hobbybastel-Raspi-Bereich sollte man meinen, dass die enorm hohe Rückläuferquoten haben müssten. Was machen die also anders?



Kommerzielles LoRaWAN-Gateway



Rückseite eines LaserCutters



KVM-over-IP: piKVM

Die Software-Trickkiste

- Man kann softwareseitig einige Maßnahmen ergreifen, um Schreibzugriffe auf die microSD-Karte zu reduzieren:
  - Logging reduzieren/in RAMDisk legen/ausschalten!
  - Alle, oder zumindest die meisten Schreibzugriffe in eine RAMDisk umleiten lassen! → Führt auch dazu, dass bei Stromausfall/hartem Reset das darunterliegende Dateisystem nicht kaputtgehen kann, weil es nur readonly gemountet ist.
  - Speichermedium nicht komplett partitionieren, freien Platz am Ende lassen (bis zu 20% heutzutage realistisch)! → Der Controller bekommt einen größeren Pool an nicht genutzten Speicherzellen.

Die Hardware-Trickkiste

- Leider sind die wenigsten Controller auf microSDs zu einem vernünftigen Wear-Leveling (also der Rotation häufig beschriebene Zelle → selten beschriebene Zelle) in der Lage, daher kommen die häufigen Ausfälle.
- Auf „größeren“ Medien wie USB-Sticks, SSDs, NVMe und eMMC sitzen aber meist bessere Controller! → Sind üblicherweise günstiger als irgendwelche „industrial grade microSDs“ zu haben, trotz vergleichbarer Qualität.
- Alle Pis ab Pi3B sind auch von USB bootfähig! Pi4 natürlich auch von USB3, für den Pi5 gibt es sogar nativen NVMe-über-PCIe-Support (Zusatzplatine notwendig), die Compute Modules gibt es mit Onboard-eMMC (was auch für industriellen Einsatz ausgelegt ist).
- Vorgängermodelle Pi1/Pi2 brauchen (micro)SD zum Booten, können aber das Rootsystem von einem USB-Medium laden. → Auch so kann man die meisten Schreibzugriffe umleiten.
- Hardware-Watchdog des Raspberry Pis aktivieren! → Löst automatischen Reset aus, wenn der „Heartbeat“ des Betriebssystems ausbleibt.

Mit der Physik verhandelt man nicht.

- Gilt nicht nur bei der Klimaerwärmung und beim Auto- und Zweiradfahren, sondern auch bei Massenspeichern – also von microSD über USB-Sticks bis zur NVMe-SSD.
- Speicherchips produzieren Abwärme:
  - Die Abwärme ist um so größer, je mehr/je länger am Stück geschrieben wird (Lesezugriffe erzeugen auch Wärme, aber nicht so viel).
  - Je wärmer der Chip wird, desto schneller altert er.
- Controller fragen daher die Chiptemperatur ab und drosseln die Zugriffsgeschwindigkeit, wenn die Temperatur steigt. → System wird träge!
- Je weniger Chips auf dem Medium verbaut sind, und umso kleiner sie sind, umso größer das Temperaturproblem:
  - geringere Abstrahlfläche → man wird die Hitze nicht so schnell los und altert den Chip
  - mit mehreren Chips könnte man durchrotieren, einer wird beschrieben, der andere darf abkühlen → hat man nur einen, geht das nicht

Cool McCool

- Nicht nur der Speicher, auch die CPU, GPU, etc. auf dem Pi selbst freuen sich über Kühlung.
- Hitzestau führt erst zu Drosselung, dann zum harten Not-Aus.
- Unbedingt Kühlkörper aufkleben!
- Möglichst große Kühlkörper wählen (besonders bei passiver Kühlung):
  - Es gibt z.B. das Gehäuse „Argon One“, bei dem das ganze Gehäuse als Kühlkörper dient! (Das „FLIRC“ ist zwar aus Aluminium, aber hohl! → Nehmt was anderes!)
  - Lüfter sind mechanisch bewegte Komponenten, fallen daher gern vorzeitig aus:
    - Besser, wenn man den Lüfter nie/nur als Reserve bei einer Hitzewelle braucht!
    - Lüfter verwenden, deren Drehzahl man über GPIO bis 0 herunterregeln kann!

Sparfuchs-Ansatz

- PCIe-NVMe + Kühlkörper oder USB-SSD sind die einfachsten Lösungen, aber auch die teuersten.
- Die eMMC-Option hat man nur bei den Compute Modules.
- USB-SSD ist ggf. auch arg sperrig, je nach Bauform.
- Wir nehmen also möglichst handelsübliche USB3-Sticks, aber:
  - Mit hohen Transferraten!
  - Mit mehreren Chips!
  - Mit großem Gehäuse, und das möglichst aus Metall:
    - Bessere Wärmeabfuhr!
    - Nachrüsten von zusätzlichem Kühlkörper möglich (Selbstklebe-Wärmeleitpad und z.B. entsprechend geformtes Kupferblech oder Aluminium-Rippenkühlkörper)!

Praxiserfahrungen USB-Sticks

Blauer Hintergrund: Aktuell schlecht erhältlich dank des KI-Hypes

- Bei uns haben sich bisher gut bewährt:
  - Kingston DataTraveler Max Type-A, 1000MB/s Lese-/900MB/s Schreibrate laut Datenblatt
    - kein Metallgehäuse → daher kein Kühlkörper möglich
    - aber groß und verteilt Zugriffe auf mehrere Chips
  - SanDisk Cruzer Extreme Pro (in der Variante mit Metallgehäuse und mittels Schieber versenkbarem USB-Port), 420MB/s Lese-/380MB/s Schreibrate laut Datenblatt
    - Metallgehäuse hat ovale Form → kein Standard-Kühlkörper möglich, man muss basteln
    - Hinweis: smartctl-Tool meldet, der Speicher sei kurz vor dem Defekt und man solle ihn austauschen. → Tatsächlich aber Fehlalarm, Sandisk verbaut einen SMART-fähigen SSD-Controller, der Reservezellen erwartet, die hat der Stick, anders als eine echte SSD, natürlich nicht. Quellen:
      - https://debianforum.de/forum/viewtopic.php?t=187219
      - https://www.wilderssecurity.com/threads/how-to-exclude-a-device-from-storage-device-health-monitoring.451357/
  - SanDisk Ultra Luxe, 150MB/s Leserate, keine offizielle Angabe zur Schreibrate, vermutlich 30-60 MB/s
    - Metallgehäuse, nahezu plane Form, gut geeignet für zusätzlichen Kühlkörper

Die Pi Foundation steuert gegen

- Das Problem mit den nervigen microSDs hat jetzt auch die Raspberry Pi Foundation erkannt, und bietet daher seit neuestem USB-Sticks aus Metall mit USB 3.0 an – sogar mit funktionierendem SMART-Support.
- Preislich liegen die aktuell bei grob 60 Euro für 256 Gigabyte bei 150 MB/Sekunde Schreibgeschwindigkeit und 32 Euro für 128 Gigabyte bei 75 MB/Sekunde Schreibgeschwindigkeit – und sie sind auch bei den üblichen Händlern schon vorrätig.
- Die Geschwindigkeit habe ich allerdings noch nicht selbst testen können. Wenn die Angaben stimmen, ist selbst der 128er schon schneller als die Modelle aus der SanDisk-Ultra-Luxe-Reihe, aber eben auch etwas teurer.
- Infos direkt bei der Pi Foundation unter https://www.raspberrypi.com/products/flash-drive/

Zwischenfazit

- Compute Module: Ein Modell mit onboard-eMMC verwenden, das ist drauf ausgelegt. Nicht den ganzen Speicher partitionieren schadet trotzdem nicht, die Schreibzugriffe auf die RAMDisk umzuleiten,wäre hier rein aus Lebenszeitsgesichtspunkten verzichtbar (aber nicht, was Stromausfall/hartes Ausschalten angeht) → insgesamt eher Kategorie „Fire & Forget“ – läuft einfach.
- „Normaler“ Raspberry Pi 3B, 3B+, 4B oder 5: USB-Stick (ab Pi4 USB3.x-Stick) verwenden, Hardware wie vorhin beschrieben auswählen, Platzreserve bei der Partitionierung vorsehen, damit der Controller Manövriermasse hat, Schreibzugriffe auf RAMDisk umleiten.
- Kühlkonzept und Hardware-Watchdog nicht vergessen!
- Man könnte auch noch eine „StromPi“-Zusatzplatine mit Akku anbringen, wenn man sich seine eigene kleine USV basteln will (und im Gehäuse noch genug Platz ist).

Und nun?

- Jetzt haben wir eine Basis, die schon recht „unkaputtbar“ ist, was äußere Einflüsse wie Temperatur und Stromausfall angeht – mit einem StromPi und Akku laufen wir sogar während kurzer Stromausfälle weiter, aber auch ohne kommen wir wieder automatisch hoch, sobald der Strom wieder da ist.
- Aber wie lösen wir das Problem, dass wir auch Updates einspielen müssen? „Draußen“, also im Feld, im Einsatz, irgendwo in der hintersten Ecke verbaut, wo man nicht mal schnell den Stecker ziehen und die microSD oder den Stick tauschen, beziehungsweise das eMMC neu flashen kann?

Nobody ever got fired for trusting IBM

- IBM hatte schon in den 1990er-Jahren in einem sogenannten „Redbook“ ein Betriebskonzept für IBM OS/2 WARP3 herausgebracht, welches mehrere Betriebssystem-Partitionen vorsah, zwischen denen mittels Bootmanager durchgewechselt werden kann:
  - Wartungspartition (nur Bootloader und ein minimales Toolset)
  - Produktionsumgebung 1
  - Produktionsumgebung 2

  → Updates werden im Wechsel eingetankt und dann der Default-Booteintrag geändert.
- Auf PCs spart man sich das heutzutage meistens, weil man die Installationsumgebung auch übers Netzwerk in eine RAMDisk booten kann, von dort vollen Zugriff auf das Massenspeichermedium hat, und dann einfach alles platt macht und neu installiert.

Zurück in die Vergangenheit

- Stellt sich raus: Dieses Konzept mit mehreren Bootumgebungen wird von der Raspberry Pi Foundation ganz offiziell auf dem Raspberry Pi unterstützt – man braucht kein alternatives Image wie NOOBS oder PINN, keine Zusatzsoftware, keinen speziellen Bootloader wie uboot oder so.
- Nur: Die Standardinstallation von Raspberry Pi OS mittels rpi-Imager bzw. dd an der Kommandozeile schießt sich dabei selbst ins Bein, weil sie keinen freien Platz für zusätzliche Umgebungen lässt. Selbst wenn man das Flag setzt, dass die Root-Partition nicht den gesamten freien Speicherplatz in Anspruch nehmen soll – die Partitionstabelle passt nicht.
- Um das zu verhindern, dürfen wir den Raspi nicht direkt von dem frisch betankten Medium booten, sondern müssen erst noch am PC ein paar Anpassungsschritte vornehmen. Das sind immer die gleichen, egal was man danach installieren will, deswegen bietet es sich an, das in ein Skript zu packen, und das als Basis für andere in einem öffentlichen Git-Repo bereitzustellen. Genau das habe ich getan. :)

Namensgebung/Partitionsaufteilung

- Die drei Umgebungen heißen hier ENV1, ENV2 und ENV3.
- Wie zu IBM-OS/2-Zeiten ist die erste Umgebung die, die nur das absolute Minimalsystem enthält.
- Die Partitionierung sieht nun so aus:
  - Partition 1 (primäre): /boot von ENV1, FAT32, 512 MB groß
  - Partition 2 (primäre): /boot von ENV2, FAT32, 512 MB groß
  - Partition 3 (primäre): /boot von ENV3, FAT32, 512 MB groß
  - Partition 4: (erweiterte):
    - Partition 5 (logische): / von ENV1, EXT4, 8 GB* groß
    - Partition 6 (logische): / von ENV2, EXT4, 8 GB* groß
    - Partition 7 (logische): / von ENV3, EXT4, 8 GB* groß
    - Partition 8 (logische): /data – wird in jedem ENV gemountet, EXT4, restlicher freier Speicherplatz*
      (Diese Partition kann man bei Bedarf aus ENV1 heraus aushängen und schrumpfen, um z.B. 20% als Reserve/Manövriermasse für den Controller zu lassen)

*Diese Größen lassen sich mittlerweile in der base_install.conf anpassen, man sollte aber genau wissen, was man tut.

Besonderheit: Netzwerkdevice

- Das Netzwerkdevice ist hier, abweichend von der Raspberry-Pi-OS-Standardeinstellung die Bridge br0.
- Das Device eth0 ist automatisch in der Bridge, so dass sich das System erst mal wie gewohnt verhält.
- Weitere Devices, die auf eth* matchen, werden in manchen Branches (z.B. single-port-printserver, multi-port-printserver) sogar schon am Ende der Installation automatisch in die Bridge br0 geworfen, um zu zeigen, dass und wie das geht.
- Warum habe ich das so gemacht?
  - Weil es Compute-Module-Baseboards mit mehr als einem Ethernet-Port gibt, und auch USB-Ethernet-Dongles existieren.
  - Auf diese Weise kann man die anderen Ports bequem in die Bridge aufnehmen, oder in weitere Bridges, oder einzeln nutzen.
  - Damit kann man z.B. einen Wifi-Access-Point oder Netzwerk-Printserver bauen, den man einfach bei einem bestehenden Gerät mit LAN-Kabel (z.B. einem IP-Telefon oder PC) „dazwischenstecken“ kann, so dass man keinen zusätzlichen Switch-Port belegt.

Github-Repository und Branches


Github-Repo

- Repository-URL: https://github.com/stefanbaur/rrpi-base-install
- Spezielle Branches:
  - main – Alle Branches holen sich Branch-übergreifende Weiterentwicklungen per merge von hier. Dieser Branch lässt sich daher als einziger nicht installieren.
  - example – ein Minimalbeispiel, welches nur das Programm „`sl`“ installiert („`apt install -y sl`“). Sehr simpel gehalten und erleichtert daher den Einstieg ins Erstellen eigener Branches.
  - minimal – im Prinzip wie der example-Branch, nur ohne „`sl`“ – das eignet sich für eine Basisinstallation, die anschließend mit Orchestrierungstools wie Ansible verwaltet wird.

Direkt nutzbare Branches (1/2)

- Folgende Branches lassen sich direkt nutzen:
  - x2goserver-kdrive-lxde – eine kleine X2GoServer-Demo-Instanz mit X2Go-KDrive, LXDE, und zwei Beispielaccounts mit SSH-Keys.
  - x2gominiclient – ein minimalistischer Desktop mit einem X2GoClient, passend zum x2goserver-Branch vorinstalliert.
  - single-port-printserver – installiert einen Daemon, der alle per IP auf Port 9100 eingehenden Daten auf einen per USB angeschlossenen Drucker wirft. Kann man auf alten Pi1/Pi2 nutzen, um sie noch sinnvoll weiterzuverwenden, oder auf einem CM-Baseboard mit zwei Ethernet-Ports, wenn man Switchports sparen will/muss und den Pi im LAN-Anschluss des PCs „durchschleift“.
  - multi-port-printserver – Erweiterte Version des obigen Branches, hier können alle USB-Ports des Pis genutzt werden, Ports 9100, 9101, usw. werden immer auf den gleichen USB-Port und somit auch dann auf den gleichen Drucker gemappt, wenn mehrere Drucker angeschlossen sind, aber ggf. nicht alle gleichzeitig eingeschaltet wurden.

Direkt nutzbare Branches (2/2)

- Folgende Branches lassen sich direkt nutzen:
  - docker-mailcow – installiert eine „dockerisierte“ Mailcow-Instanz (Mailserver mit IMAP/SMTP und Webmail).
  - docker-paperless-ngx – installiert eine „dockerisierte“ Paperless-ngx-Instanz (Dokumentenmanagementsystem).
  - docker-euro-office – installiert eine „dockerisierte“ EuroOffice-Instanz (zur Anbindung an z.B. NextCloud – für Demo-Zwecke auch standalone nutzbar).
  - wifi-to-upstream-lan-accesspoint – spannt ein WLAN auf, mit dem man sich dann Richtung LAN verbinden kann.

Weitere Branches

- Work in Progress:
  - grannophone – hier findet die Software-Entwicklung für mein Senioren-Videotelefon „Grannophone“ statt.
  - lan-to-upstream-wifi-bridge – dockt sich an ein vorhandenes WLAN an, und ermöglicht, Geräte, die nur LAN- oder USB-Ports haben, ins WLAN einzuklinken.
- Geplant:
  - nas-proxy – dazu gedacht, Home-NAS-Systeme abzukapseln, wenn z.B. ...
    - der Hersteller keine Firmware-Updates mehr bereitstellt.
    - die Firmware „fishy“ Dinge tun versucht (I’m looking at you, Western Digital – heimlich einen VPN-Tunnel zur Herstellerfirma aufbauen, obwohl die Funktion in der WebGUI vom Besitzer abgeschaltet wurde, ist nicht cool!).
    - die Übertragungsrate bei verschlüsselten Protokollen wie scp/sftp/rsync/sshfs im Vergleich zu NFS3 oder „altem“ SMB übelst einbricht, weil die CPU zu schwach für kryptographische Funktionen ist.

Wie setzt man das System nun auf?

- Für die Grundinstallation, so wie sie auch von der Raspberry Pi Foundation vorgesehen ist, gibt es mittlerweile im Git-Repo https://github.com/stefanbaur/rrpi-base-install/ im Ordner rpi-imager-2.0.x-manual/de eine bebilderte, deutschsprachige Schritt-für-Schritt-Anleitung von mir – und ich unterstelle mal, dass ihr den rpi-imager alleine installiert bekommt. Tipps dazu stehen in der README.md im obersten Verzeichnis meines Repos.
- Wer von euch den Begriff cloud-init schon mal gehört hat: Das Image unterstützt das mittlerweile ebenfalls, und der rpi-imager nutzt es auch.
  Ihr könnt den rpi-imager also umgehen, und eure eigenen cloud-init-Konfigurationsdateien direkt in das Image werfen, wenn ihr euch damit auskennt.
- Vielleicht baue ich mein Skript auch mal so aus/um, dass es direkt cloud-init nutzt.
  Das ist das übliche Zeitproblem in Verbindung mit „tut doch auch so“ ...

Die Skriptkonfiguration

- Wir klonen das Repository und checken erst mal den main-Branch aus:

```
mkdir -p ~/git && cd ~/git
git clone https://github.com/stefanbaur/rrpi-base-install.git
cd rrpi-base-install
git branch -a              # → zeigt die Liste aller verfügbaren Branches
git checkout main          # den main-Branch auswählen
```

- Die für alle Branches einheitliche „Magie“ steckt in der Datei „`base_install.conf`“. Wenn ihr euch keinen eigenen Branch anlegen wollt, könnt ihr sie in eine Datei „`base_install_custom.conf`“ kopieren, diese hat immer Vorrang vor der „`base_install.conf`“ aus dem Repo und bleibt beim Branch-Wechsel stehen.

- Was gleich wichtig wird: Ihr braucht irgendwo ein temporäres Verzeichnis mit mindestens 15GB freiem Speicherplatz.
  → Voreingestellt ist „`/tmp`“, aber das lässt sich ändern.

base_install.conf (Kurzfassung)

```
# WICHTIG:
# Eine optionale "base_install_custom.conf" hat immer Vorrang vor dieser Datei und wird von "git pull" nicht überschrieben

# Anpassen, wenn /tmp bei euch keine 15GB frei hat
export TEMPIMAGEDIRROOT="/tmp"

# USB-Ethernet-Gadget einschalten (Debug-Feature, schauen wir uns heute an)
export DEBUG_VIA_USBETH="true"
export FIRSTTHREEOCTETS="10.0.0"
export NETMASK="255.255.0.0"

# SPI-TFT-Display einschalten (Debug-Feature, im Kielux-Vortrag erklärt)
export DEBUG_VIA_SPI="true"
export DEBUG_VIA_SPI_DTO_DOWNLOAD="https://example.com/file.zip"
export DEBUG_VIA_SPI_CONFIG_PARAM="dtoverlay=piscreen"
export DEBUG_VIA_SPI_CONFIG_ADD_PARAMS="hdmi_cvt 480 320 60 6 0 0 0"
export DEBUG_VIA_SPI_CMDLINE_ADD_PARAMS="fbcon=map:10"

# Nummer des GPIO-Pins, den man auf Masse ziehen muss, damit beim nächsten Start zwingend ENV1 gebootet wird,
# egal, was in der Boot-Datei steht (Details in nächster Folie).
# GPIO 21 ist der letzte Pin rechts am Ende Richtung USB-/Ethernet-Port; Masse liegt direkt daneben, der letzte Pin links
export GPIO_FORCE_ENV1="21"

# Manchmal hat man nur ein kleineres Speichermedium und braucht gar keine so große Root-Partition, z.B. für die Printserver-
# Branches. Dann kann man mit diesem Parameter kleinere Root-Partitionen erzwingen.
export OVERRIDE_ROOTFS_MAXSIZE="3G"

# Normalerweise wird /data der restlichen verfügbaren Speicherplatz zugewiesen.
# Mit diesem Parameter kann man das deckeln – z.B. für das erwähnte Wear Leveling.
export OVERRIDE_DATA_MAXSIZE="4G"
```

Hardware-Boot-Umschalter

- Mit einem GPIO-Pin, den man in der base_install.conf definieren kann, kann man den Raspberry Pi dazu zwingen, beim nächsten Neustart das ENV1 zu booten – selbst wenn in der Konfigurationsdatei des Bootloaders etwas anderes steht. Dazu muss man ihn nur beim Neustart auf Masse ziehen.
  (Hinweis: Es ist nicht sauber dokumentiert, wie lange der Pin auf Masse bleiben muss. Am besten abwarten, bis die ACT-LED das Herzschlag-Blinken – Poch-Poch, Pause, Poch-Poch, Pause – zeigt.)
- Der Fantasie sind da praktisch keine Grenzen gesetzt, vom einem Jumper, den man bei Bedarf aufsteckt, über einen normalen Schalter/Taster, einen versenkten Taster, den man nur mit einer aufgebogenen Büroklammer erreicht, bis zum Schlüsselschalter ist alles drin. Auch ein Schalter mit einem Seilzug, wie bei einer altmodischen Tisch- oder Stehlampe wäre denkbar, um den Reset bei einem nur umständlich erreichbar verbauten Raspi bequem aus der Ferne auszulösen, oder ein Fotosensor in Verbindung mit einer Glasfaser, falls man Streckenlängen überbrücken muss, bei denen man mit einem Kabel kein zuverlässiges Signal mehr hinbekommt, bzw. wo Überspannungsgefahr bei Blitzschlag droht (Raspi auf der Kirchturmspitze, der mit der Webcam das Storchennest filmt und per WLAN streamt).
- Da in ENV1 nur die Minimalinstallation steckt, und man es daher selten updaten muss, stehen die Chancen gut, hier noch auf eine bootfähige und per SSH aus der Ferne erreichbare Umgebung zu treffen, wenn man sich die andere(n) zerlegt hat.

„example“-Branch

- Wir wechseln für unser erstes Beispiel in den Branch „example“:
  `git checkout example # den example-Branch auswählen`
- Hier schauen wir uns die Shellskripte im Verzeichnis „`./templates`“ an:
  ```
  cd templates
  ls -1 *.sh
  ```
- Dort befinden sich:

```
autostart.sh # wird bei der Installation in jedem ENV und bei jedem Start
#              ausgeführt – *kürzt sich aber nach der Installation selbst zusammen*

ENV1-custom-install.sh # Wird bei der Installation in ENV1 ausgeführt, *WIRKT*
#                        aber auch in ENV2 und ENV3 – da diese aus ENV1 geklont werden

ENV2-custom-install.sh # wird nach dem Klonvorgang nur in ENV2 ausgeführt

ENV3-custom-install.sh # wird nach dem Klonvorgang nur in ENV3 ausgeführt
```

- Wiederum: Wollt ihr nicht euren eigenen Branch pflegen, könnt ihr eure Änderungen statt in „`./templates`“ einfach in „`./custom`“ ablegen – dort haben sie Vorrang vor den Templates und werden nicht von einem `git pull` überschrieben.

ENV1-custom-install.sh

- Im „example“-Branch sieht das recht einfach aus.
- Ein `-e` führt dazu, dass bei Fehlern abgebrochen wird, so dass wir keine „verbogene“ Installation bekommen, sondern sauber auf einen Fehler laufen.
```
#!/bin/bash -e
```
- Ein paar erklärende Kommentare, was man hier warum tun bzw. besser lassen lassen sollte.
```
# THIS SCRIPT WILL BE EXECUTED INSIDE THE CHANGEROOT, NO NEED TO CALL chroot HERE
# run this right before umounting, so there's no apt clean call destroying our work
```
- Hier nun der Aufruf von `apt-get install`, mit einem `echo` darüber, damit man auch sieht, wo das Skript gerade ist, und warum es diese Pakete herunterladen muss (an dieser Stelle werden sie durch das `-d` nur heruntergeladen, aber noch nicht installiert) – sie sind nämlich Voraussetzung für ein funktionierendes Overlay-Dateisystem. Dieses aktivieren wir später für die Root-Partitionen von ENV1, ENV2, und ENV3 – nur /data bleibt verschont.
WICHTIG: Wie schon auf der vorangegangenen Folie erklärt, werden alle Pakete, die man hier auflistet, hinterher auch in ENV2 und ENV3 zu finden sein, und alle Schritte, die man hier sonst noch ausführen lässt, werden auch in ENV2 und ENV3 Auswirkung haben.
```
# download the packages required for overlayroot, so you don't need internet access to
activate it
echo "Downloading required packages for overlayroot, so it can be installed later - even
without internet access."
apt-get install -d -y cryptsetup cryptsetup-bin overlayroot
```

ENV2/ENV3-custom-install.sh

- Die Skripte für ENV2 und ENV3 unterscheiden sich bezüglich der Paketliste deutlich von dem von ENV1, aber praktisch kaum voneinander.
- Ein `-e` führt wieder dazu, dass bei Fehlern abgebrochen wird, so dass wir keine „verbogene“ Installation bekommen, sondern sauber auf einen Fehler laufen.
  ```
  #!/bin/bash -e
  ```
- Auch hier gibt es wieder den erklärenden Kommentar, was man hier besser lassen lassen sollte.
  ```
  # THIS SCRIPT WILL BE EXECUTED INSIDE THE CHANGEROOT, NO NEED TO CALL chroot HERE
  ```
- Und wieder der Aufruf von `apt-get install`, mit einem `echo` darüber, damit man auch sieht, wo das Skript gerade ist, und welche Pakete es zur Installation vorbereiten will (auch an dieser Stelle werden sie durch das `-d` nur heruntergeladen, aber noch nicht installiert).
  Da wir im „example“-Branch sind, ist es nur ein Paket, nämlich „`sl`“ – die Steam Locomotive.
  In diesem Branch unterscheiden sich die beiden Dateien nur dadurch, dass in `ENV3-custom-install.sh` statt dem Text „ENV2“ dann „ENV3“ steht.
  Natürlich wäre es aber möglich, ENV2 und ENV3 an dieser Stelle mit unterschiedlichen Paketen zu betanken, z.B. weil man ENV3 mit zusätzlichen Debug-Tools versehen will.
  ```
  echo "Downloading the 'sl' package into ENV2, so it can be installed later - even
  without internet access."
  apt-get install -d -y sl
  ```

autostart.sh – 1/2

- Die `autostart.sh` wird nach `/data` kopiert und von dort ausgeführt. Das mag riskant erscheinen, aber da wir `/data` im Normalfall mit „sync“ mounten, ist auch hier das Risiko eines Datenverlusts durch ein kaputtes Dateisystem bei plötzlichem Crash oder Stromausfall gering.
- Diese Datei ist so groß, dass wir sie im Rahmen dieses Vortrags nicht sinnvoll im Detail durchgehen können, daher nur zwei Folien dazu.
- Während der Erstinstallation/-konfiguration sorgt sie dafür, dass die vom rpi-imager/cloud-init gesetzten Änderungen auch wirklich eingespielt sind, bevor unsere Anpassungen laufen – anschließend löscht sie das cloud-init Paket.
- Außerdem ist in ihr ein Mechanismus eingebaut, der die ACT-LED auf Herzschlag-Blinken setzt, sobald das System fertig in ENV1 gebootet ist (und auf 50:50-Blinken in ENV2, und Dauerleuchten in ENV3) – damit man weiß, wann man den Hardware-Boot-Umschalter wieder loslassen/zurückstellen kann.

autostart.sh – 2/2

- Hier im „example“-Branch installiert/aktiviert sie anschließend nur noch das Overlay-Dateisystem, und konfiguriert es so, dass `/data` eben kein Overlay, sondern dauerhaft schreibbar ist. In anderen Branches wird vor der Aktivierung des Overlay-Dateisystems z.B. auch noch weitere Software installiert, der Hardware-Watchdog aktiviert und/oder alle Ethernet-Interfaces in die Bridge aufgenommen.
- Zum Abschluss wird in ENV1 der Apt-Cache geleert, der Bootloader auf ENV2 gesetzt, und ein Neustart ausgelöst.
- Nun wiederholt sich das Spiel für ENV2 und ENV3 – wobei hier dann z.B. noch Konfigurationsdateien für die Dienste geschrieben werden und die vorher heruntergeladenen Pakete pro ENV installiert werden – im Example-Branch also „`sl`“.
- Am Ende der Installation von ENV3 wird dann der Block aus der `autostart.sh` gelöscht, der nur für die Installationen notwendig war, der Bootloader auf ENV2 gesetzt, ein Reboot ausgelöst, und wir sind in Produktion.

Besonderheiten bei x2goserver

- x2goserver erstellt (wenn noch nicht vorhanden) ...
  - den Ordner `rrpi-base-install/custom/x2goserver/`
  - darin wiederum die Unterordner `user1/` und `user2/`
  - und in diesen dann jeweils die SSH-Schlüsselpaare `id_ed25519` und `id_ed25519.pub`.
  - Die öffentlichen Schlüssel werden dann ins installierte x2goserver-System kopiert.
- Da `/data` in diesem Branch „`sync`“ gemountet ist, wird das im Betrieb eher kein Performance-Wunder.
  Entfernen des Parameters auf eigene Gefahr – denkt daran, auch `autostart.sh` liegt auf `/data` . Es ist zwar sichergestellt, dass das System nicht stehen bleibt, wenn es die Datei nicht findet (weil `/data` sich aufgrund von Fehlern nicht mounten lässt oder weil ihr die Datei gelöscht habt), aber so ganz unkaputtbar ist das System dann halt nicht mehr.

Einspieler



X2GoServer

Besonderheiten bei x2gominiclient

- Wichtig: Wenn man x2goserver und x2gominiclient installieren will, dann immer zuerst x2goserver, damit x2gominiclient die SSH-Private-Keys findet!
- Man kann die Dateien unterhalb von `rrpi-base-install/custom/` natürlich auch vorab manuell anlegen!
  → So kann man seine eigenen, angepassten Versionen einspielen und die Reihenfolge der Installation ist egal.
- x2gominiclient erstellt (wenn noch nicht vorhanden):
  - den Ordner `rrpi-base-install/custom/x2goclient/`
  - darin die Config-Dateien `printing, sessions, settings, .xsession` (landen im installierten x2gominiclient-System in `/home/[DefaultUser]/.x2goclient/-[DefaultUser]` ist, was ihr im Imager/in cloud-init als Benutzername angegeben habt)
  - sowie ein angepasstes Startscript `x2goclient` (landet im installierten x2gominiclient-System dann in `/usr/local/bin/`)
- Die privaten Schlüssel aus `rrpi-base-install/custom/x2goserver/` landen im installierten x2gominiclient-System in `/home/[DefaultUser]/.ssh/user{1,2}/`
  → bei abweichenden Usernamen muss man evtl. noch die `autostart.sh` anpassen

Einspieler



X2GoClient

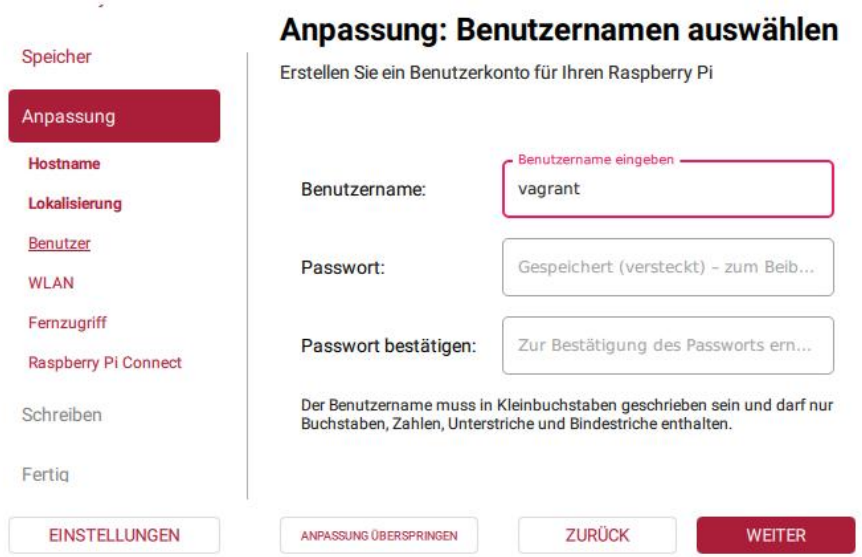
„minimal“-Branch und Ansible

- Der Minimal-Branch entspricht dem Example-Branch, allerdings ohne das beispielhaft installierte „sl“-Paket.
- Ansible ist ein Open-Source-Automatisierungswerkzeug zur Administration von Computern („orchestrieren“ → kurz gesagt, einen bestimmten Soll-Zustand auf einem oder mehreren gleichartigen Systemen hart erzwingen).
- Wenn man die Raspberry-Pi-OS-Grundinstallation mit dem Raspi-Imager geeignet anpasst (host- und username, sowie SSH-Public-Key), und anschließend den Installer für den „minimal“-Branch laufen lässt, kann man den Pi, sobald sich das Image auf dem Pi fertig selbst installiert und konfiguriert hat, sehr einfach mit Ansible orchestrieren.
- Wir folgen also zunächst der Rpi-Imager-Anleitung aus dem Repo, und passen die drei Stellen an, die in den nachfolgenden zwei Folien gezeigt werden.

Rpi-Imager Teil 1/2

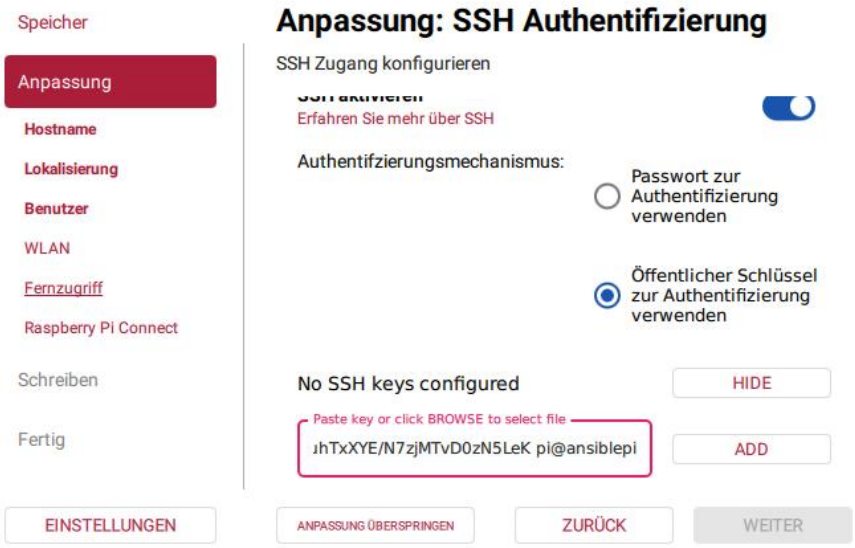
Wir installieren heute beispielhaft einen LAMP-Stack (Linux, Apache, MariaDB, PHP) via Ansible. Dazu wählen wir im Rpi-Imager einen passenden Host-Namen und setzen den Benutzernamen auf den, den Ansible später verwenden wird:

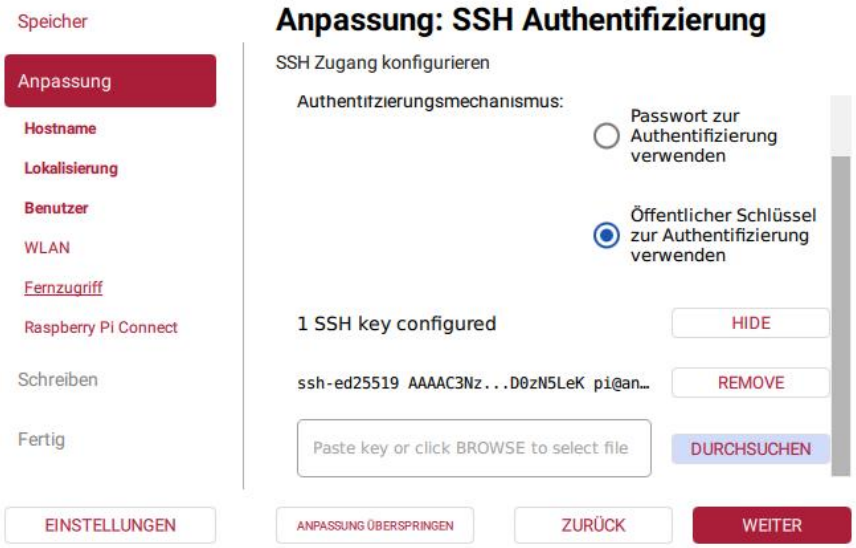




Rpi-Imager Teil 2/2

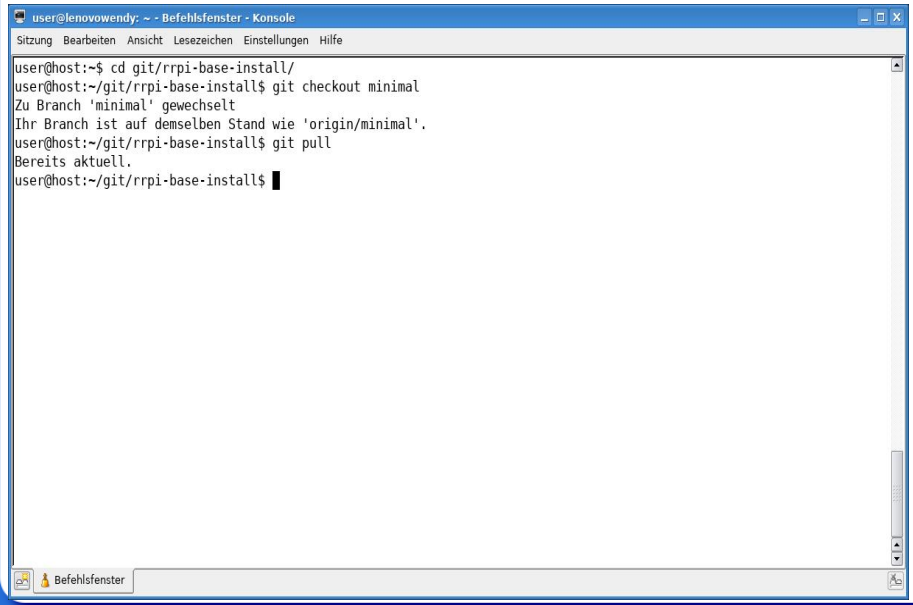
Beim Schritt „Anpassung: SSH Authentifizierung“ wählt man die Variante mit dem öffentlichen Schlüssel aus. Danach muss man gegebenenfalls noch einen Button „SHOW“ neben der Meldung „No SSH keys configured“ anklicken. Danach fügt man den Public Key seines Ansible-Accounts ein und bestätigt ihn mit „ADD“, bevor man auf „WEITER“ klicken kann.

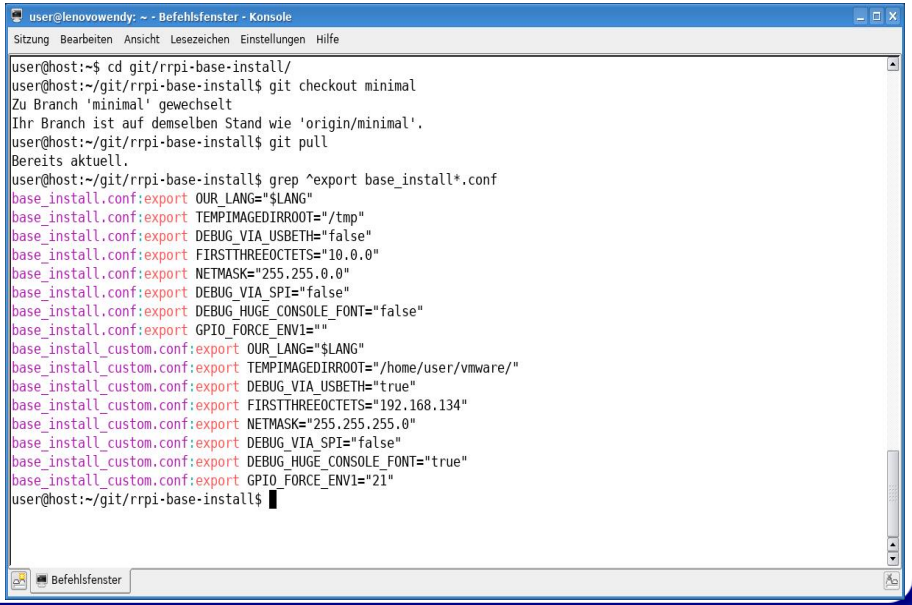




Branch auswählen/Variablen prüfen

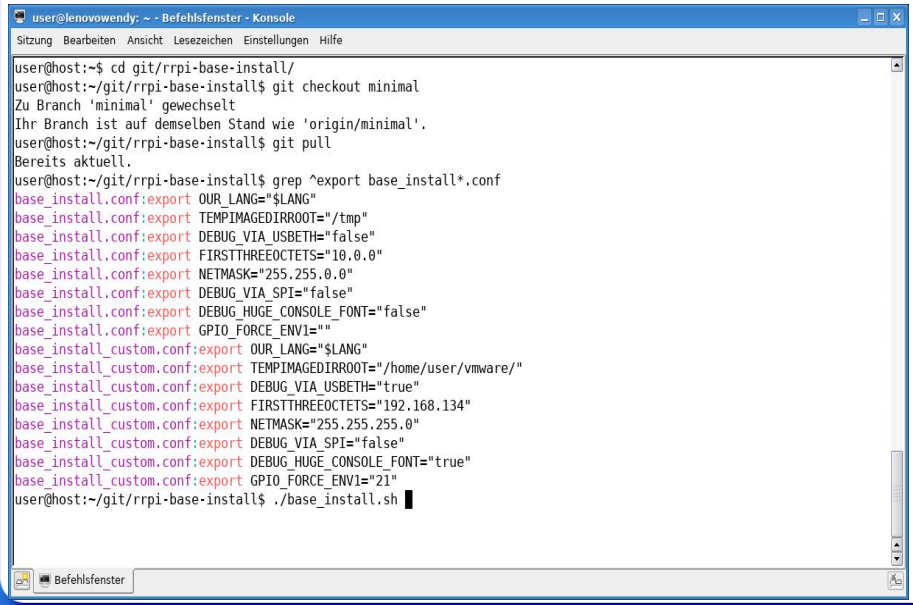
- Wenn der Imager sein Werk getan hat, zieht man den Stick kurz ab, und steckt ihn wieder ein.
- Dann wechselt man in das Repo und checkt den „minimal“-Branch aus.
- Als nächstes lässt man sich die Config-Variablen anzeigen und passt sie gegebenenfalls in der base_install_custom.conf an.

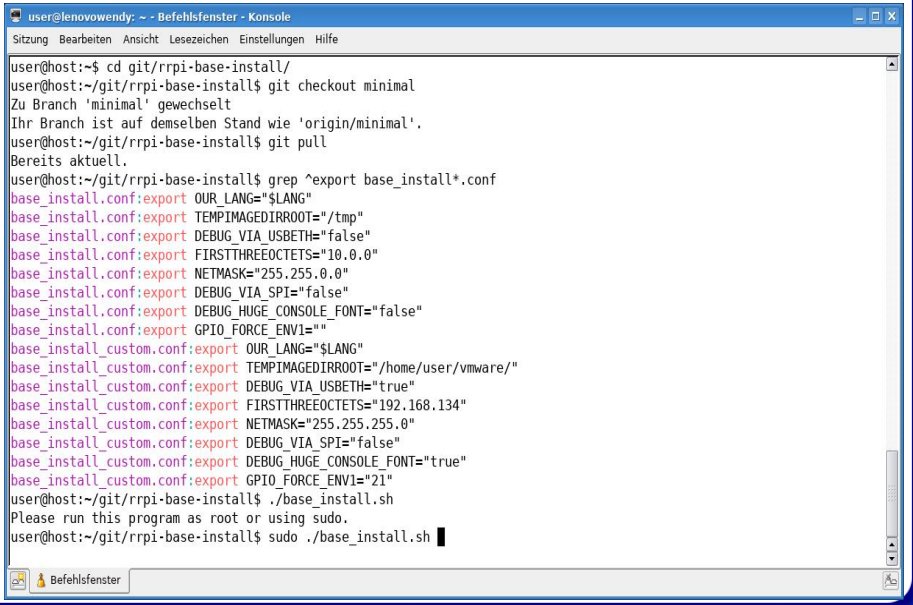




Start frei!

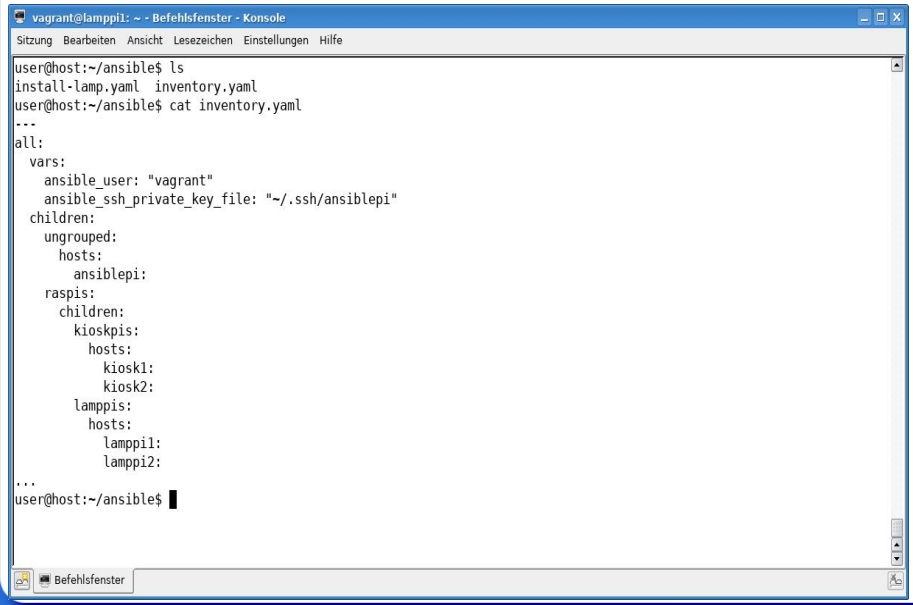
- Jetzt startet man das Installationsskript ...
- ... und wenn man das als root, bzw. mit sudo macht, dann klappt das auch. ;)

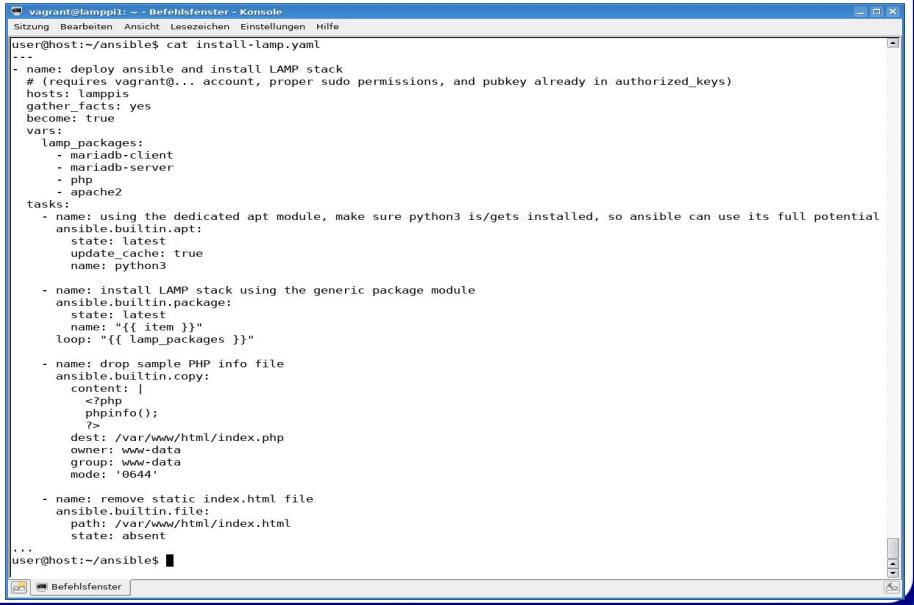




Der Ansible-Teil

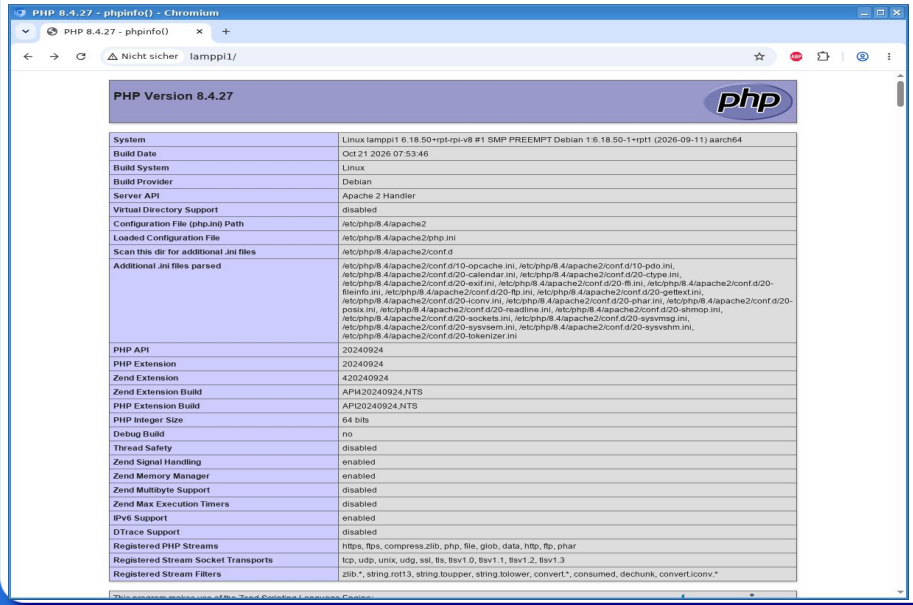
- Auf dem Pi (oder PC), der unseren Ansible-Control-Host darstellt, brauchen wir natürlich neben dem Ansible-Paket selbst (`sudo apt install ansible`) und dem entsprechenden SSH Private Key auch noch ein Inventory und ein Playbook, um den LAMP-Stack zu installieren.
- Die Dateien packe ich euch ebenfalls ins Repo, unter `./ansible-examples`

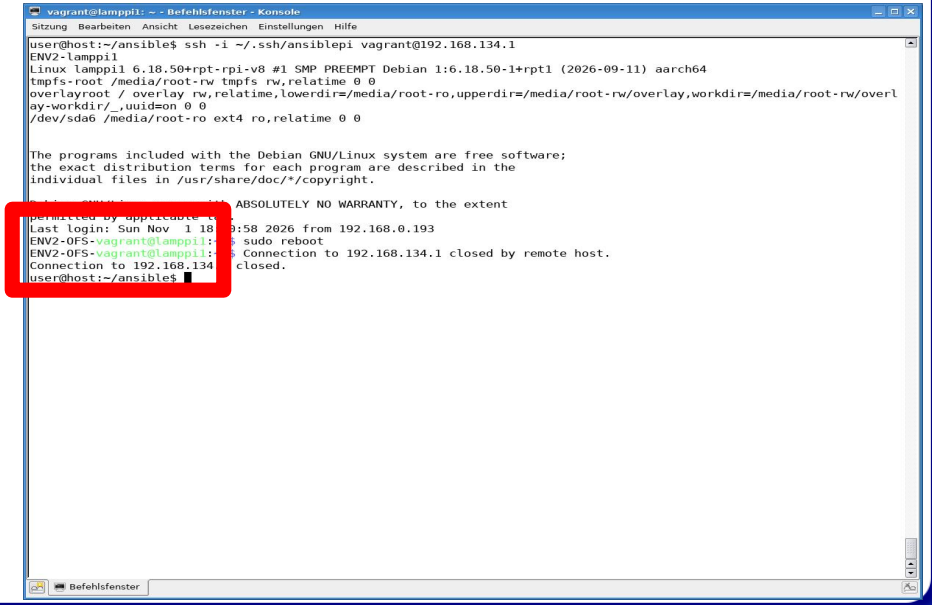




Mit einem Wisch ist alles weg!

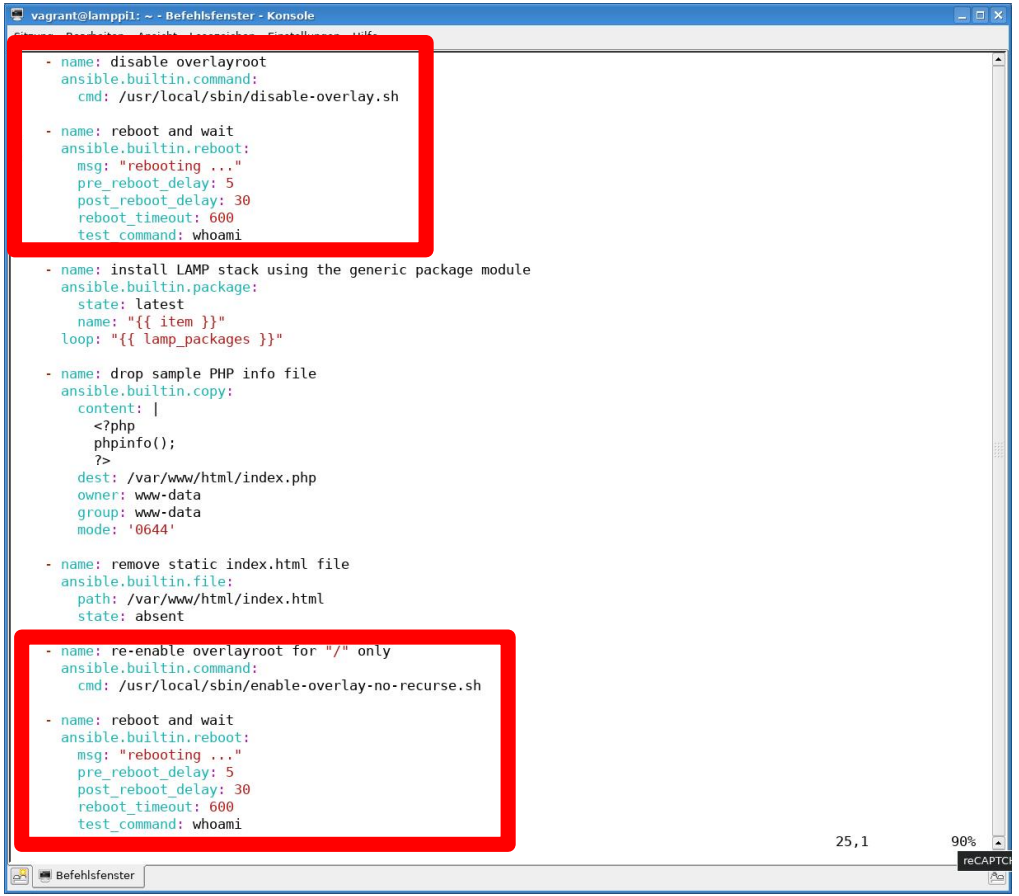
- Der Befehl `ansible-playbook -i ./inventory.yaml -l lamppi1 install-persistent-lamp.yaml` bügelt uns nun den LAMP-Stack auf den Pi – was wir im Browser mittels `http://lamppi1` überprüfen können.
- Hat alles geklappt, sehen wir unter der Adresse eine PHP-Info-Seite.
- Da gibt es allerdings noch ein subtiles Detail, welches nur auffällt, wenn wir uns mal direkt auf dem Pi einloggen: **Nach einem Reboot ist die ganze Installation wieder futsch!**
- Der Grund ist, dass das Overlay-Root-Filesystem aktiv war, erkennbar an dem „OFS“ im Prompt.





Anhaltenden Erfolg sicherstellen

- Es gibt mehrere Wege, die Installation persistent zu bekommen.
- Am einfachsten geht es, indem man die Installationsschritte im Playbook mit den Befehlen zum Aus- und Wiedereinschalten des Overlay-Root-Filesystems „kapselt“. Damit das funktioniert, ist allerdings jedes Mal ein Neustart notwendig.
- Das entsprechend erweiterte Playbook findet ihr natürlich auch im Repo. :)



Nächste X2Go-Events

Noch nicht angemeldet/noch keine Zusage:

- 28.03.2027 – 29.03.2027 Chemnitzer LinuxTage

Termin/Ort noch offen:

- „X2Go: The Gathering/The Spread 2026“
- orca e.V. Jahreshauptversammlung 2026

Wiki: https://wiki.x2go.org/doku.php/events:start

Event-
Wikiseite

X2Go lebt vom Mitmachen

- Helfer gesucht!
- X2Go kann immer zwei Dinge von euch brauchen:
  - Zeit/KnowHow – auch von Nicht-Programmierern!
  - Geld/Hardware/Dienstleistung: Man kann ...
    - über den orca e.V. (gemeinnützig) eine zweckgebundene Spende an X2Go leisten
    - eine der Firmen im Projekt mit einer konkreten Aufgabe (Bugfix, Feature Request) beauftragen

Spenden/Aufträge

- Für Spenden haben wir mehrere WirWunder/Betterplace-Seiten (Events, Infrastrukturkosten, ...) direkt auf der Wiki-Startseite: https://wiki.x2go.org
- Firmen, die für Aufträge zur Verfügung stehen: https://wiki.x2go.org/doku.php/0spnn5 (null-spnn-fünf)

Spendenlink

Liste der Supportfirmen

Vielen Dank für das Interesse!
Sehen wir uns in Chemnitz?

Spendenlink

Liste der Supportfirmen